

TTS-ptbr — A trilha completa do projeto

Unflat Studio

18 de junho de 2026

CONTENTS

TTS-ptbr — A trilha completa do projeto	
Da aposta arquitetural ao Treino 1: tudo o que construímos, em linguagem de quem está aprendendo o campo	
Como ler este documento	
PARTE I — A visão e a aposta	
1. O objetivo: uma “Maya brasileira”	
2. O negócio: por que isso vira produto (o wedge)	
3. A correção fundadora: áudio é a espinha, texto é paralelo	
PARTE II — Os conceitos que você precisa dominar	
4. Anatomia de uma conversa por voz	
5. Como uma máquina “fala”: codec neural, tokens de áudio, RVQ	
5.1 Por que tokenizar áudio	
5.2 RVQ — Residual Vector Quantization (a pilha de codebooks)	
5.3 Tokens semânticos vs. acústicos — o pulo do gato	
6. Os dois modelos centrais: CSM (a voz) e Moshi (a conversa)	
6.1 O CSM — Conversational Speech Model (a “boca”)	
6.2 O Moshi — o full-duplex de verdade (a “espinha”)	
7. Como se molda esses modelos: finetune, LoRA, RL	
7.1 Finetune, LoRA, adapter	
7.2 Zero-shot, in-context, âncoras	
7.3 CPT e quantização	
7.4 SFT, DPO, GRPO — a escada do RL (sobretudo para emoção e interatividade)	

8. Emoção e sotaque: por que não há atalho mágico

8.1 Por que “task vectors / activation steering” não funcionam no nosso spine

8.2 O sotaque decompõe em fonético + prosódico

8.3 G2P e GOP/MDD — as ferramentas do sotaque

8.4 Os rótulos em três camadas (a decisão concreta de dataset)

PARTE III — A engenharia reversa da Maya

9. O que o CTO da Sesame revelou (podcast a16z, mar/2025)

10. O raio-X do stack (OSINT)

11. O cenário competitivo e o quadrante vazio

PARTE IV — A trilha do software: o que já construímos

12. As três trilhas + a transversal (o mapa)

13. Fundação: os datasets e o problema dos dados pt-BR

O kit de gravação: a ciência de gravar a sua voz

14. Trilha A — A Voz: a receita de dois estágios e a jornada de treino

14.1 A descoberta que ancorou tudo: “voz é barata, língua é o investimento”

14.2 Estágio A — ensinar português ao CSM

14.3 Estágio B — a sua voz por cima da BASE-PT

14.4 Os gotchas de treino — os bugs caros que viraram receita

14.5 O Treino 1 — o que ele provou

15. Trilha M — Maya-BR v0: a cascata, peça por peça

16. Trilha B — A Conversa (Moshi): por que e quando

17. Transversal: eval, o compasso (rate_app), a esteira e os aprendizados

17.1 O eval harness — como medimos se a voz ficou boa

17.2 O rate_app — o compasso do projeto

17.3 A esteira RunPod — a 2ª pista de treino

17.4 Os aprendizados como ativo

PARTE V — Onde estamos e para onde vamos

18. O que o Treino 1 provou — e o que falta

19. O roadmap priorizado e o plano de GPU

20. Os princípios de trabalho

Fecho: a tese em uma respiração

Glossário

TTS-ptbr — A trilha completa do projeto

DA APOSTA ARQUITETURAL AO TREINO 1: TUDO O QUE CONSTRUÍMOS, EM LINGUAGEM DE QUEM ESTÁ APRENDENDO O CAMPO

Documento-insumo para podcast (NotebookLM). Escrito para o fundador — engenheiro de ML, iniciante em modelos de áudio e conversação em tempo real. Termos técnicos em inglês; explicações em português, com analogias. Estado do projeto em **18 de junho de 2026**.

Como ler este documento

Este texto é uma travessia. Ele começa na **pergunta** (“por que estamos construindo uma voz conversacional brasileira?”), passa pelos **conceitos** que você precisa dominar para entender as decisões (o que é um codec neural, o que é full-duplex, por que emoção não é um botão), atravessa a **engenharia reversa** do nosso ídolo competitivo (a Maya, da Sesame), percorre a **trilha do software** que já existe no repositório — peça por peça, treino por treino — e termina em **onde estamos e para onde vamos**.

Não é um resumo executivo. É longo de propósito: a ideia é que, ao final, você consiga explicar com suas próprias palavras como uma máquina aprende a falar com a sua voz, por que o sotaque “de gringo” é o problema número um, e o que cada arquivo do nosso repositório está fazendo ali. Há um **glossário** no fim — mas os termos também são explicados na primeira vez que aparecem.

Uma convenção honesta que herdamos da nossa própria pesquisa: quando um número ou afirmação é **medido/verificado**, ele aparece com confiança; quando é **estimativa ou hipótese aberta**, isso é dito. Esse rigor é parte do método do projeto.

PARTE I — A visão e a aposta

1. O objetivo: uma “Maya brasileira”

O projeto tem um norte único e exigente: construir **o melhor sistema de fala conversacional em português brasileiro**, no nível da **Maya** — a voz de IA da startup americana **Sesame** que viralizou em 2025 por soar absurdamente humana. Não é “mais um TTS”. A diferença está em três palavras que vamos desdobrar ao longo do documento:

- **Conversacional** — não é locução de bula ou GPS. É fala que responde *dentro de uma conversa*, herdando o clima, o ritmo e a emoção do que veio antes.
- **Tempo real** — latência de resposta com alvo **p50 < 800 ms** (e ideal de 200–300 ms), com **turn-taking** (a dança de quem-fala-quando) e **barge-in** (você corta a IA no meio e ela cala na hora).
- **Expressiva e brasileira** — emoção controlável e, o nosso diferencial autoral, **sotaque carioca** com até cinco sub-variações (carioca médio, zona sul, cria, surfista, interior).

O **dataset-semente** — a matéria-prima inicial — é a **sua própria voz**, Pedro: carioca, gravada de forma dirigida. Isso não é um detalhe sentimental; é uma escolha estratégica que volta o tempo todo neste documento, porque **dado de voz emocional e carioca, com licença que permita uso comercial, simplesmente não existe pronto no mundo**. Quem grava a sua, sai na frente.

O nome do repositório (“TTS-ptbr”) é, na verdade, modesto demais para o escopo. Como diz a nossa própria constituição: *“apesar do nome, o escopo é uma plataforma de voz conversacional full-duplex, não um motor clássico de TTS.”* O produto final consome voz em lugares concretos já mapeados: uma demo no site da Unflat, instalações artísticas (a “vitrola de IA” do SambaCore já usou a sua voz clonada na ElevenLabs), um balcão de atendimento público (“Fala Cidadão”), SAC agêntico para empresas, e uma “maquininha” física para varejo e eventos.

2. O negócio: por que isso vira produto (o *wedge*)

Antes de uma linha de código, o projeto passou por um exercício de *office hours* (a metodologia de YC de pressionar a demanda real). A conclusão é importante porque **ancora todas as decisões técnicas num cliente que já existe**.

O **wedge** (a fatia mais estreita por onde se entra no mercado) é a **voz-assinatura para trabalho experiencial e cultural**. O arquétipo é a **Mariclea / Sesc**: uma cliente recorrente da Unflat (três anos, R\$ 350 mil+ em projetos) que, num ciclo, especificou um microfone com voz de IA para uma ativação (~R\$ 40 mil, seis dias por semana, 1–2 h de fala por dia) e a Unflat **recusou a função puramente pelo custo da ElevenLabs**. Ela ficou aberta a fechar no próximo ciclo com uma solução própria da Unflat. Isso é, literalmente, **um pedido de compra adiado travado neste produto**.

A matemática do problema é brutal e ensina o moat: um único dia (~8 h ≈ 480 min) de um agente de voz na ElevenLabs consome **a cota mensal inteira do plano Pro (~R\$ 520)** — cerca de R\$ 500/dia numa instalação real, ~19% do custo de uma ativação de R\$ 40 mil. O comprador cultural compra **anualmente, com orçamento fixo, e rejeita por princípio custo variável repassado**.

O **moat** (o fosso competitivo, a vantagem difícil de copiar) é **ter um modelo pequeno (1B) e próprio de pt-BR**. Servir esse modelo custa estimados **R\$ 0,02–0,12 por minuto** — algo como **10 a 40x abaixo da ElevenLabs**. E é estrutural: o modelo é pequeno e é nosso, então o custo marginal é uma fração da API paga. A ElevenLabs (valuation na casa dos bilhões, entrando no Brasil) é **puro custo variável** e não consegue oferecer preço fixo sem canibalizar a própria receita.

O requisito de produto que emergiu disso — e que o cliente ditou — é **preço fixo anual + medição de cota + modo fail-closed** (a IA trava de vez quando a cota acaba, zero conta variável). Só é economicamente possível por causa do moat.

A “motion” é **A → B**: (A) vender a voz como uma linha de preço fixo embutida nos contratos de ativação da Unflat agora; (B) padronizar uma oferta de “voz-assinatura” para o segmento experiencial, financiada pela A. Governo, maquininha e API são **expansão**, não o wedge.

3. A correção fundadora: áudio é a espinha, texto é paralelo

Há um documento curto no repositório, `phase0/RETHINK.md`, que registra a decisão arquitetural mais importante do projeto — e ela nasceu de uma correção sua, Pedro.

No início, o assistente tinha ancorado em “Maya = híbrido com texto no core”. Você corrigiu: o que a Sesame faz de excepcional é **conversação em áudio**; o texto é paralelo, nunca a espinha. Essa correção foi verificada contra as **fontes primárias** (os papers) e virou lei do projeto:

- **Moshi (o paper, arXiv 2410.00037), verbatim:** *“operar puramente no domínio do áudio já produz resultados convincentes”*. O fluxo de texto que o Moshi carrega — o **Inner Monologue** — é um “andaime” paralelo, alinhado no tempo, que *“aumenta a qualidade linguística”* e é **ablável** (você pode removê-lo num experimento para medir o quanto ele contribui). Áudio é a espinha; texto é um auxílio paralelo.
- **CSM/Sesame:** o CSM *“não consegue gerar texto”*. Ele é um gerador de fala de uma única elocução, condicionado no **áudio** da conversa anterior. A sensação de “vida” da Maya vem desse condicionamento por áudio; um LLM separado, no bastidor, só fornece as *palavras*.

A consequência, que organiza toda a engenharia: **a emoção, a prosódia, a voz e o turn-taking vivem no próprio modelo de fala conversacional. O texto (STT/LLM/Inner-Monologue) é bastidor/paralelo, nunca o núcleo**. Essa é a razão de o projeto apostar no **Moshi** como espinha de longo prazo e usar o **CSM** como o componente de voz — e nunca o contrário.

Por que isso importa para você, fundador: modelos de linguagem (LLMs) têm um viés gravitacional de puxar tudo para o texto. Essa correção é o anticorpo contra isso. Toda vez que uma decisão parece colocar a transcrição no caminho crítico da emoção, é sinal de alerta.

PARTE II — Os conceitos que você precisa dominar

Esta parte é a “aula”. Ela constrói, do zero, o vocabulário de áudio e conversação em tempo real. Se você dominar esta parte, o resto do documento (a trilha real do software) lê-se sozinho.

4. Anatomia de uma conversa por voz

Quando você fala com a Maya e ela responde, é tentador imaginar **um cérebro único** que ouve voz e devolve voz. Na prática (inclusive na Maya — confirmado pelo próprio CTO da Sesame, ver Parte III), o que existe é uma **cascata** (*cascade*): vários sistemas especializados em fila, cada um fazendo bem o seu pedaço.

```
Seu áudio → [VAD] → [ASR] → texto → [LLM] → texto de resposta → [TTS] → áudio falado
```

Vamos por cada peça:

- **VAD — Voice Activity Detection.** Um detector de “tem gente falando agora?”. Recebe um pedacinho de áudio e devolve uma probabilidade de aquilo ser fala (vs. silêncio, ar-condicionado, ruído de rua). É o gatilho de mais baixo nível.
- **ASR — Automatic Speech Recognition.** O transcritor. Converte sua voz em texto. *Incremental/streaming* significa transcrever **enquanto** você fala, cuspindo hipóteses parciais — para o resto da cadeia começar a trabalhar antes de você terminar. (No nosso protótipo atual o ASR roda *depois* do turno; mais sobre essa diferença adiante.)
- **LLM — Large Language Model.** O “cérebro/roteirista”. Recebe o texto transcrito e decide **o que dizer**. No nosso desenho ele é **plugável** e fica no **bastidor** (*backstage*) — pode ser Gemini, Maritaca/Sabiá, um modelo local. Importante: na cascata, o LLM **vê só o texto** — toda a emoção do seu áudio (ironia, raiva, cansaço) se perde na transcrição.
- **TTS — Text-to-Speech.** A “boca/dublador”. Transforma o texto da resposta em áudio. No caso especial do CSM, ele recebe também o **áudio** da conversa, e por isso fala com a emoção certa.

Três conceitos de conversação que separam “assistente de voz” de “conversa de verdade”:

- **Full-duplex** — ouvir e falar **ao mesmo tempo**, como num telefonema. Os dois canais de áudio ficam abertos o tempo todo. **Half-duplex** é o walkie-talkie: um fala, o outro espera. (O nosso app local usa half-duplex por padrão em caixa de som, por causa do eco — explicado na Parte IV.)
- **Barge-in** — interromper o agente falando por cima, e ele cala na hora. Numa conversa humana, se você interrompe e o outro continua mais um segundo, soa robótico e irritante.
- **Backchannel** — aqueles “uhum”, “aham”, “sei”, “caraca”, “sério?” que o ouvinte solta **enquanto o outro fala**, sem tomar o turno. Sinalizam “tô te acompanhando”. Um TTS que nunca ouviu backchannel só sabe monologar.

E o divisor de águas arquitetural mais importante de todos:

- **Cascata (cascading) vs. native audio (áudio nativo).** A cascata é a linha de montagem em estações separadas (acima). O *native audio* é um único cérebro que recebe e devolve **tokens de áudio** diretamente, intercalando texto e som — sem a tradução para texto puro no meio, então a prosódia e a emoção não se perdem. A Maya e a nossa aposta (Moshi/CSM) são da família native audio. A cascata é o **piso de latência** e um plano-B sério; o native audio full-duplex é o **teto** de naturalidade.

5. Como uma máquina “fala”: codec neural, tokens de áudio, RVQ

Aqui está o conceito mais denso e mais importante para um engenheiro de ML que vem do mundo do texto. Vale ir devagar, porque ele destranca tudo.

5.1 POR QUE TOKENIZAR ÁUDIO

Um transformer (a arquitetura por trás dos LLMs) opera sobre **sequências de símbolos discretos** — tokens. Texto já é naturalmente discreto (palavras, subpalavras). Áudio **não é**: é um sinal contínuo, dezenas de milhares de números por segundo (a 24 kHz, são 24 mil amostras/segundo). Para um transformer “pensar” em áudio como pensa em texto, primeiro é preciso **discretizar** o som — virar uma sequência finita de “letras de áudio”.

Esse trabalho é feito por um **codec neural** (*neural audio codec*): um par **encoder/decoder** treinado para **comprimir áudio em tokens discretos** e **reconstruir o áudio** a partir deles. O codec da nossa família de modelos chama-se **Mimi** (da Kyutai). O **encoder** do Mimi pega a forma de onda e a transforma numa sequência curta de tokens; o **decoder** pega os tokens que o modelo gerou e os transforma de volta em som audível. **O modelo de fala modela tokens; o Mimi traduz entre tokens e som.**

5.2 RVQ — RESIDUAL VECTOR QUANTIZATION (A PILHA DE CODEBOOKS)

Como se transforma um pedacinho de som contínuo num token? Com **Vector Quantization (VQ)**: você tem um “dicionário” fixo de vetores-protótipo (um **codebook**), acha o protótipo mais parecido com o seu pedaço de áudio, e guarda só o **índice** dele (um número). É como arredondar uma cor qualquer para a cor mais próxima numa paleta de 256 cores — você perde precisão, mas ganha um símbolo discreto.

O problema: um único codebook não tem resolução para reconstruir fala de alta fidelidade. O protótipo mais próximo sempre erra um pouco; essa diferença chama-se **resíduo (residual)**. A solução é **RVQ — Residual Vector Quantization**, que **empilha** vários codebooks:

1. O primeiro codebook aproxima o áudio e guarda o índice. Sobra um resíduo (o erro).
2. Um **segundo** codebook quantiza esse resíduo. Sobra um resíduo menor.
3. Um **terceiro** quantiza o resíduo do segundo, e assim por diante.

Resultado: cada **frame** de áudio não é *um* token, mas uma **pilha de N códigos** (um por codebook), cada nível refinando o erro do anterior. É como descrever um número por aproximações sucessivas — “é mais ou menos 3” (nível 1), “3,1” (nível 2), “3,14” (nível 3). Quanto mais níveis, mais fiel a reconstrução — ao

custo de mais “pinceladas” por frame.

O Mimi opera a **12,5 Hz** — apenas 12,5 frames por segundo, ou seja, **80 ms por frame** ($1 \div 12,5 = 0,08$ s). Essa compressão brutal (de 24.000 amostras/segundo para 12,5 frames/segundo, por nível) é o que torna viável um transformer rodar áudio em tempo real. Guarde o número **80 ms/frame**: ele reaparece como a latência mínima do Moshi (Parte 6) e como a granularidade de corte do nosso barge-in (Parte IV).

5.3 TOKENS SEMÂNTICOS VS. ACÚSTICOS — O PULO DO GATO

Os níveis do RVQ no Mimi não têm o mesmo papel. Essa separação é o detalhe que diferencia o Mimi de um codec comum:

- **Tokens semânticos (semantic tokens)** — carregam o **conteúdo linguístico**: *o que* está sendo dito, a estrutura fonética e de significado. São o “esqueleto inteligível” da fala. No ecossistema CSM/Mimi, o **codebook zero** (o primeiro nível) concentra esse papel mais estrutural.
- **Tokens acústicos (acoustic tokens)** — carregam o **detalhe sonoro fino**: timbre, textura, a identidade que faz a voz soar como *aquela pessoa*. São os refinamentos dos níveis superiores do RVQ.

Resumo operacional: **semantic = o que se diz e a estrutura; acoustic = como soa, o timbre, a identidade**. Isso é diretamente acionável no nosso projeto: **clonar a sua voz** é em boa medida ensinar o modelo a produzir os tokens **acústicos** certos (o seu timbre carioca), enquanto **adaptar a língua** (pt-BR, fonologia, sotaque) mexe mais no terreno **semântico/estrutural**. Não é coincidência que a nossa receita separe **adaptação de língua** (Estágio A) de **voz** (Estágio B) — são camadas diferentes do problema, alinhadas a esses dois tipos de token.

6. Os dois modelos centrais: CSM (a voz) e Moshi (a conversa)

6.1 O CSM — CONVERSATIONAL SPEECH MODEL (A “BOCA”)

O **CSM** é o modelo de voz que a Sesame abriu em março de 2025, na versão **csm-1b** (1 bilhão de parâmetros), licença **Apache-2.0**. É o nosso clonador de voz. Ele tem **três peças** que vale separar mentalmente:

1. **Backbone (espinha interna do CSM)** — um transformer **autoregressivo** baseado em **Llama** (a família de LLMs da Meta). É o “cérebro sequencial”: processa o histórico (texto + áudio dos turnos anteriores) e, frame por frame, prevê o código mais estrutural do próximo frame de áudio. *Autoregressivo* = gera um pedaço de cada vez, e cada novo pedaço é realimentado como entrada para gerar o próximo (igual a um LLM gerando texto token a token, só que aqui os “tokens” são de áudio).

2. **Depth decoder (decodificador de profundidade)** — a partir do esqueleto que o backbone produziu, ele percorre a **profundidade dos codebooks RVQ** e gera os códigos acústicos restantes daquele frame, refinando resíduo após resíduo. Por isso “depth”: ele caminha na profundidade dos codebooks dentro de um mesmo frame. Há um truque de eficiência chamado **compute amortization**: o depth decoder é treinado em só **1/16 dos frames** (enquanto o codebook zero é treinado em todos) — barateia o treino sem destruir a qualidade.
3. **Codec Mimi** — traduz áudio ↔ tokens (seção 5). Vem da Kyutai, licença **CC-BY-4.0** (já aceita pelo projeto).

O conceito que faz a Maya soar viva é o **audio-conditioning**: o CSM recebe como entrada o **áudio dos turnos anteriores** da conversa, não só o texto da frase atual. Analogia: a diferença entre um dublador lendo uma fala isolada num papel e um ator que acabou de ouvir o colega e responde no mesmo clima — o segundo entrega a deixa “viva” porque herda o estado emocional e o ritmo. A entonação da resposta **não nasce do nada**; ela é gerada *a partir de* como a conversa estava soando.

E um ponto que costuma confundir: **o CSM gera fala SEM gerar texto**. Ele *recebe* texto (a frase a falar + o histórico) e *produz áudio* (a pilha de tokens RVQ de cada frame, que o Mimi decoder vira som). Ele **não decide o que dizer** — é um **renderizador de fala**, não um roteirista. É por isso que o CSM é **componente de voz**, não **spine**: ele é a boca, não o cérebro que decide o conteúdo. No nosso sistema, quem decide o conteúdo é o LLM (na cascata) ou o Moshi (no spine).

Sobre os **tamanhos**: o CSM existe em 1B, 3B e 8B, mas **só o 1B é aberto**. A Sesame fechou — não há CSM-2, não há API, não há 3B/8B públicos. Isso, na prática, é uma sorte operacional: o 1B é pequeno o suficiente para finetunar em GPU barata (LoRA cabe numa T4 grátis do Colab). E há um fato que o próprio CTO confirma e que vamos ver na Parte III: **“even the 1B is very good”** — a naturalidade base já está resolvida no 1B; a escala compra outra coisa.

6.2 O MOSHI — O FULL-DUPLEX DE VERDADE (A “ESPINHA”)

O **Moshi** (Kyutai, pesos **CC-BY-4.0**) é o único grande modelo aberto que é **full-duplex de verdade**. Em vez de revezar turnos, ele modela **dois streams de áudio paralelos simultaneamente**: um é o que *você* está dizendo (entrada), o outro é o que *o Moshi* está dizendo (saída). Os dois rodam no mesmo relógio, frame a frame, sem nunca parar. Por isso ele consegue, em princípio, **overlap** (falar junto), **backchannel** e **interrupção** de forma **nativa** — essas coisas não são casos especiais que ele precisa detectar; são o estado normal de ter dois canais abertos.

A coerência linguística vem do **Inner Monologue**: além dos streams de áudio, o Moshi carrega um **stream de texto alinhado no tempo**, em paralelo, que funciona como o “pensamento” que guia a fala (com o texto prevendo um pouco à frente do áudio). É paralelo (mantém o full-duplex) e abelável (dá para medir sua contribuição).

A **latência** do Moshi é de **~200 ms** (na prática, 160–200 ms em GPU local) — e esse número não é arbitrário: cai direto da estrutura do Mimi (12,5 Hz, 80 ms/frame; a latência mínima de um modelo que gera quadro a quadro é um punhado de frames). Compare com uma cascata bem-feita, que entrega resposta

voz-a-voz em ~700 ms–1 s. O Moshi é a única arquitetura onde overlap/backchannel sequer são possíveis — e o **mesmo codec Mimi** é compartilhado entre Moshi e CSM, o que torna o nosso dado pt-BR **fungível** entre os dois (áudio tokenizado com Mimi serve para ambos).

Três peças provam que dá para mexer no Moshi sem esperar a Kyutai:

- **moshi-finetune** (Apache-2.0): código oficial de LoRA e full finetune. LoRA pede ~39,6 GB de VRAM (cabe numa H100; aperta numa A100-40).
- **J-Moshi** (japonês) e **LLM-jp-Moshi** (Apache-2.0): provam que **transferência de língua funciona** na família Moshi — e pode sair com licença permissiva. A receita: adaptar vocabulário → CPT (continued pre-training) com pseudo-estéreo → finetune com diálogo estéreo real.
- **moshika-rl-seamless** (CC-BY-4.0, jun/2026): o **RL de interatividade** pronto — exatamente o pós-treino que íamos ter que inventar (mais na seção 7.4).

7. Como se molda esses modelos: finetune, LoRA, RL

Você vem do mundo de ML, então alguns destes termos são familiares — mas vale fixar o vocabulário específico de fala.

7.1 FINETUNE, LORA, ADAPTER

- **Finetune** — continuar o treino de um modelo pronto com **seus** dados, para especializá-lo (ex.: ensinar pt-BR com a sua voz ao CSM).
- **LoRA — Low-Rank Adaptation** — finetune barato: você **congela** o modelo original e treina só umas matrizes pequenas, de baixa dimensão (*low rank*), acopladas “por fora”, que aprendem só o **delta** de comportamento. Analogia: em vez de reescrever um livro inteiro, você cola post-its de correção nas margens. Muda o comportamento com uma fração ínfima dos parâmetros — barato, rápido, reversível. O **adapter** é o arquivo resultante (no nosso repo, vive em `models/stage_b_final_adapter/`).
- **Full finetune** — treina o modelo inteiro. Mais pesado, mais caro, mas às vezes necessário para mudanças estruturais grandes (como adaptar uma língua nova).

7.2 ZERO-SHOT, IN-CONTEXT, ÂNCORAS

- **Zero-shot / in-context** — sem treino nenhum: o modelo imita a partir de exemplos no prompt. Clonar voz com 8 s de referência é in-context learning. Os exemplos de referência são as **âncoras**. Um achado concreto nosso: no CSM em 4-bit, **use ≥ 3 âncoras** — medimos spk-sim subir de **0,618 (1 âncora) para 0,973 (3 âncoras)**.

7.3 CPT E QUANTIZAÇÃO

- **CPT — Continued Pre-Training** — re-treino pesado de língua (pega o modelo pré-treinado e continua pré-treinando em dados da nova língua). É o nosso fallback caro, se LoRA não bastar.
- **Quantização (quantization)** — comprimir os pesos (16-bit → 8/4-bit) para rodar mais rápido/leve, com possível perda de qualidade. No seu M2, 4-bit + 3 âncoras = quase tempo real sem perder o timbre.

7.4 SFT, DPO, GRPO — A ESCADA DO RL (SOBRETUDO PARA EMOÇÃO E INTERATIVIDADE)

Esta é a escada de treino para “comportamentos” que não têm uma resposta certa única que você copia, e sim um comportamento que você **recompensa**:

- **SFT — Supervised Fine-Tuning** — treino supervisionado clássico, o “copie estes exemplos” (pares texto+rótulo → áudio desejado).
- **DPO — Direct Preference Optimization** — alinhamento por **preferência** sem treinar um modelo de recompensa separado. Você monta pares (**preferido, rejeitado**) — “esta versão soou melhor que aquela” — e o DPO empurra o modelo na direção do preferido. Em vez de ensinar uma regra, você mostra dois takes e diz “esse, não aquele”.
- **GRPO — Group Relative Policy Optimization** — um método de RL (o mesmo que ficou famoso no treino de modelos de raciocínio). Para cada contexto, o modelo gera um **grupo** de respostas, cada uma recebe uma **recompensa** (reward), e o modelo é empurrado na direção das que ficaram acima da média do grupo. Não precisa de um “modelo crítico” separado e pesado — a referência é o próprio grupo. É o que dá os maiores saltos de “realismo percebido”.

A `moshika-rl-seamless` da Kyutai é exatamente isso: GRPO com **recompensas por eixo** (pausa, turn-taking, backchannel, interrupção) derivadas de conversa humana real, mais um **LLM-judge** para não degradar o conteúdo. É o blueprint do nosso pós-treino futuro (Fase 5).

O risco do RL: reward hacking. RL otimiza o que você *mede* — se a métrica é furada, o modelo aprende a *enganar a métrica* em vez de soar bem. Por isso a regra de ouro do projeto: **GRPO multi-reward só depois que o eval harness pt-BR existir.** “Sem reward confiável, RL só amplifica viés.”

8. Emoção e sotaque: por que não há atalho mágico

Este é um dos insights mais valiosos e contraintuitivos do projeto, e ele determina meses de roadmap.

8.1 POR QUE “TASK VECTORS / ACTIVATION STEERING” NÃO FUNCIONAM NO NOSSO SPINE

Existe uma técnica sedutora chamada **activation steering**: você roda exemplos de uma emoção (digamos, “raiva”), observa as ativações internas do modelo, calcula uma **direção** que representa “raiva”, e na hora da inferência **soma esse vetor** às ativações para deslizar o output naquela direção — *sem treinar nada*. É como descobrir um knob invisível chamado “raiva” no painel de mixagem e girá-lo ao vivo.

O problema: **isso só funciona em modelos flow-matching** (F5-TTS, CosyVoice2), **não no spine autoregressivo** (Moshi, CSM, Qwen3-TTS). A razão é profunda: **num modelo AR, a prosódia/emoção emerge da continuação autoregressiva — não é um embedding separável**. Cada token de áudio é condicionado em todos os anteriores; a “raiva” não vive numa direção limpa e estável que você possa somar, está distribuída no estado dinâmico que se constrói token a token. Empurrar uma direção fixa nas ativações de um AR não dá um knob — dá ruído e degradação.

Isso foi inclusive **provado** para o nosso caso: a UNESP (arXiv 2606.05367, jun/2026) mostra que “nenhum operando antes do x-vector admite controle emocional por aritmética linear” em TTS baseado em LM. **Conclusão para a decisão:** a rota mágica “controle de emoção sem regravar via vetor” está fora da mesa para o spine. O caminho real é **dataset multi-emoção gravado** (a sua voz, atuando os estilos). Não é um atalho que estamos preguiçosos para tomar — é o único caminho que funciona.

Cuidado para não confundir: **activation steering** (mexe nas ativações em runtime — não pega no AR) é diferente de **task vector / accent vector** (mexe nos pesos, resultado de treino — esse *sobre-vive* no AR, porque você entrega ao modelo um conjunto de pesos “já com sotaque” e ele continua gerando normalmente). Cada LoRA já é um delta de pesos — perfeito para virar um “vetor” componível. É a rota tecnicamente elegante para as 5 sub-variações cariocas, mas é “v2”, quando houver eval.

8.2 O SOTAQUE DECOMPÕE EM FONÉTICO + PROSÓDICO

Sotaque não é uma coisa só. Ele tem (pelo menos) dois ingredientes que vivem em lugares diferentes do pipeline:

- **Camada fonética** — *quais* sons você produz e *com que regras*. O carioca chia o /s/ no fim de sílaba (“eskola” → “eshkola”), tem africadas (ti → “tchi”, di → “dji”), o /r/ de coda como fricativa ([χ]/[h]). Isso é, em boa parte, uma questão de **mapeamento grafema→fonema** e de **regras fonológicas**.
- **Camada prosódica** — a *melodia* e o *tempo*: entonação, duração das vogais, ritmo, o “swing” da frase. Mesmo pronunciando os mesmos fonemas, a prosódia carioca já entrega de onde a pessoa é.

Por que decompor importa: **cada camada tem uma alavanca diferente**. A fonética você ataca pela **entrada** (fonemizando o texto — G2P, abaixo) e por léxico/regras. A prosódica você ataca por **dado dirigido** (você gravando com o ritmo certo) ou por task vectors. Tratar “sotaque” como bloco monolítico leva a soluções que mexem na coisa errada.

8.3 G2P E GOP/MDD — AS FERRAMENTAS DO SOTAQUE

- **G2P — Grapheme-to-Phoneme** — “grafema para fonema”: a função que converte texto na sua pronúncia em símbolos fonéticos (IPA). A ortografia é uma “compressão com perdas” da fala: o “x” pode soar /z/ (exceção), /ks/ (táxi), /ʃ/ (xícara). **Fonemizar a entrada antes do TTS** é entregar a pronúncia mastigada, sem ambiguidade — reduz o “sotaque de gringo” e protege a inteligibilidade. É exatamente onde a camada fonética do sotaque carioca pode ser injetada de forma limpa.
- **GOP / MDD — Goodness of Pronunciation / Mispronunciation Detection (and Diagnosis)** — o **nome acadêmico do “sotaque de gringo”**. GOP é um score que mede, **fonema a fonema**, quão bem cada som foi pronunciado vs. o esperado (um professor de idiomas dando nota a cada som). MDD detecta a pronúncia errada e diz *qual* fonema saiu errado e *como*. O detalhe-chave: essas técnicas **localizam o fonema errado no tempo** — alinham o áudio aos fonemas esperados e apontam *em que instante* e *em qual fonema* a pronúncia desviou. É literalmente o “onde está o sotaque de gringo, segundo a segundo” — e por isso vira tanto um **eval de sotaque** quanto um sinal localizado para o próximo treino.

8.4 OS RÓTULOS EM TRÊS CAMADAS (A DECISÃO CONCRETA DE DATASET)

A interface de controle de emoção convergiu, em 2026, para **descrição em linguagem natural** (estilo) + **tags** (eventos). Por isso a gravação da sua voz deve nascer com **três camadas de rótulo**, que habilitam as três interfaces de controle de uma só vez:

1. **Tag de evento** (discreta, inline) — `<risada>`, `<suspiro>`, “uhum”. Captura eventos pontuais precisos no tempo.
2. **Caption em linguagem natural** (por trecho) — ex. “irônico, acelerando no final”. É a interface vencedora de 2026 para estilo global.
3. **Tag de variação** (de sotaque) — ex. `[carioca:cria]`. Liga a camada dialetal.

Ninguém tem tags paralinguísticas em pt-BR — então `<risada>`, `<suspiro>`, “uhum”, “é...”, “tipo...” gravados na sua voz são um diferencial barato e imediato.

PARTE III — A engenharia reversa da Maya

A Maya é a nossa régua. Felizmente, ela é muito mais transparente do que parece — porque a Sesame deixou pistas públicas demais. Esta parte reúne o que aprendemos com (a) uma entrevista longa do CTO deles e (b) uma investigação OSINT (open-source intelligence: detetive com fontes públicas, não espionagem).

9. O que o CTO da Sesame revelou (podcast a16z, mar/2025)

O CTO **Ankit Kumar** deu uma entrevista de 1h41 ao podcast da a16z. É a fonte conceitual mais rica do projeto, porque é o próprio construtor **admitindo a arquitetura real** — inclusive o que ela *não* faz. As revelações, com o conceito por trás:

1. A Maya viral é uma CASCATA. Verbatim: *"we do use transcription in the product, in the demo... mostly it's about speed."* O fluxo é ASR incremental → LLM (só texto) → CSM audio-conditioned. **Implicação direta:** a nossa cascata Maya-BR v0 (Whisper-streaming + LLM pt-BR + CSM finetunado) **não é uma aproximação pobre — é literalmente A arquitetura da Maya de mar/2025.**

2. O LLM NÃO ouve o seu áudio (e eles admitem a lacuna). *"The current demo does not hear the user from the perspective of their paralinguistic... emotional tone... transcription misses that entirely."* A **paralinguística** (tudo que a voz comunica além das palavras — a mesma frase “que ótimo” pode ser entusiasmo ou sarcasmo) se perde na transcrição. A nuance crucial: o **LLM** (decide o quê dizer) é surdo à emoção, mas o **CSM** (decide como falar) é audio-conditioned e *ouve* todo o áudio da conversa. Por isso a Maya parece “espelhar” sua emoção na hora de responder, mesmo decidindo o conteúdo a partir de texto seco. Esse espelhamento é “o diferencial barato que separa ‘TTS bom’ de ‘presença’”.

3. Sub-500 ms é “a bunch of systems engineering” — não mágica de modelo. A fluidez não vem de um modelo genialmente rápido; vem de **engenharia de sistemas**: hiper-otimizar cada estágio, **pipelining** (sobrepor o trabalho dos estágios, como uma linha de montagem), **pré-computação** e **caching**. *"50 milliseconds here, 50 milliseconds there can really add up."* Dez descuidos de 50 ms estouram o orçamento de meio segundo. **A ordem de prioridade que isso impõe ao nosso projeto: sistemas > persona > modelo.**

4. “Even the 1B is very good”: escala compra a cauda longa, não a naturalidade. Eles treinaram 1B, 3B e 8B. A **naturalidade base** já está resolvida no 1B; escalar **não** deixa a voz “mais natural” — compra a **long tail**: os casos raros e contextuais. As duas sondas que provam isso: **homograph disambiguation** (palavras escritas igual mas pronunciadas diferente conforme o sentido — “lead” chumbo vs. liderar; em pt-BR: *sede* d’água vs. *sede* da empresa, *gosto* verbo vs. substantivo) e **pronunciation consistency** (manter a mesma variante de pronúncia que foi dada no exemplo). *"The contextual things are what we care the most about."* **Implicação:** insistir no CSM-1B é defensável com a palavra do próprio CTO — você abre mão da cauda longa de pronúncia, não da naturalidade.

5. Turn-taking = heurísticas FORA do modelo. *"These models today do not model the structure of the conversation at all... you still need some other set of models or systems that drive the conversation."*

Quando responder, quando se calar, quando aceitar interrupção — tudo isso é dirigido por um conjunto **separado** de sistemas que são, hoje, "models involved in some heuristics". **Implicação:** turn-taking não precisa esperar full-duplex. VAD + regras + um classificador leve de fim-de-turno é **literalmente o que a Sesame faz**.

6. Como eles medem qualidade (a escada de evals, porque o WER saturou). O WER (Word Error Rate — taxa de palavras erradas ao transcrever de volta o áudio gerado) "saturou" — todo TTS bom já tira nota quase perfeita, então a métrica não distingue mais. A escada deles, do quantificável ao qualitativo:

- **Sondas de pronúncia** (homógrafos, consistência de variante, nomes próprios) — *product-centric*: nasce de uma dor real ("quando a IA fala seu nome errado, dói").
- **Arena de preferência** — humanos comparam dois outputs lado a lado (estilo Chatbot Arena), Elo.
- **Win-rate vs. continuação humana real** — pegam um trecho de conversa humana, cortam, e geram duas continuações: a que o **humano real** disse e a que o **modelo** gera; um avaliador cego escolhe. A régua mais alta possível: "é tão bom quanto uma pessoa de verdade naquele momento?".
- **Loop qualitativo (hill-climbing)** — *"trying it, feeling it."* É o princípio que fecha: *"if you have your evaluations too divorced from the product experience, you might not find these product-feeling qualitative upsides."* **Adotamos essa escada inteira no nosso eval pt-BR.**

7. O roadmap deles, e citam o Moshi nominalmente. Curto prazo: fundir tudo num **transformer único** que entende áudio, gera texto e gera fala (adicionando *understanding* antes de *generation*, porque é mais fácil). Longo prazo: **full-duplex, decisão a cada ~100 ms** — e *"there's one called Moshi that's kind of interesting."* O CTO da Sesame **valida a nossa Trilha B**. E o problema em aberto que ele admite ("como inicializar arquiteturas duplex mantendo o conhecimento de LLMs") é **exatamente o que o moshi-fine-tune contorna** — a dor que trava a Sesame é a porta que o Moshi abre para um time pequeno.

10. O raio-X do stack (OSINT)

Sem acesso interno, dá para montar o retrato do sistema fechado da Maya com três fontes de pistas públicas: os **forks do GitHub** da org `SesameAILabs`, as **vagas de emprego** e os **papers dos pesquisadores** contratados. Os achados:

A cronologia dos forks é o roadmap deles, lido de fora. 2024 = VAD (silero) + ASR (whisper) + inferência → a cascata veio primeiro. Início de 2025 = serving sglang + watermark + release do CSM. Meados de 2025 = visão (óculos), treino em escala, limpeza de dados, e um fork do **moshi** (estudo de full-duplex real). Fim de 2025 = post-training (torch tune). **A maioria dos forks tem ZERO diff próprio** (são espelhos puros) — confirmando que *"a mágica é orquestração + dados, não forks secretos"*.

O único fork com engenharia própria visível é o `sglang` (o engine de serving do LLM), com 7 commits. Dois são reveladores:

- **logit_bias** — empurrar para cima ou banir tokens específicos na saída do LLM. Ferramenta clássica para **forçar/banir tokens de controle** (tags de estilo, formatação JSON).
- **O commit de 1 linha do próprio CTO:** trocar `asyncio.sleep(1)` por `asyncio.sleep(0.02)` no loop que processa **aborts de geração em voo** → reagir em **20 ms em vez de 1 segundo**. Isso é a “digital” de um sistema que **cancela a geração do LLM no meio o tempo todo** — exatamente o que barge-in e “pivotar no meio da frase quando chega resultado de busca” exigem.

Juntando os achados (logit_bias + clamp anti-`-inf` para não quebrar JSON + a lib `outlines` de geração estruturada): **o LLM da Maya emite JSON estruturado, com tokens controlados, cancelável a 20 ms o tempo todo**, separando “o que falar” de “chamadas de ferramenta”. É o mecanismo do “pivotar no meio da frase”.

Watermark silentcipher em produção. Eles tornaram o watermarking (marca d’água inaudível de 40 bits) **tensor-nativo e residente em GPU** — para marcar áudio **inline no pipeline em tempo real**. O código de referência **não tem caminho de geração sem watermark**, e roda em **MPS (no Mac M2!)**. Para nós: barato, pronto, **adotar no dia 1** — e já fica *compliance-by-design* com o PL 1460/2026 (que vai exigir watermark + consentimento para réplica de voz).

Zero patentes. Busca no USPTO por “Sesame AI”: nada. *“A vantagem deles não está protegida por patente — está em dados (1M h), engenharia e velocidade.”* **A Maya é replicável.** O que nos separa dela é quantidade de dados e qualidade de orquestração — coisas que se constroem, não um muro jurídico. E a receita de dados deles (pipeline Parler-TTS de anotação sintética + EmoCtrl de pseudo-rotulagem) é **toda open e replicável em pt-BR** — inclusive um atalho de sotaque: um classificador linear sobre embeddings de language-ID acerta **86% com 53 sotaques**.

11. O cenário competitivo e o quadrante vazio

Onde batemos nos outros, nos sete eixos (arquitetura, latência, emoção, clonagem, full-duplex, sotaque, aberto/fechado):

- **Sesame/Maya** — native audio (Llama + decoder Mimi), emoção entre turnos aprendida dos dados, clonagem in-context. CSM-1B é aberto; **a Maya completa é fechada**.
- **ElevenLabs** — o líder de mercado e a qualidade/preço a bater. Mas é **cascading**, o **v3 expressivo não streama**, e **full-duplex só existe em research** (eles **admitem** o trade-off confiabilidade↔expressividade). pt-BR default é “neutro SP” (paulista). 100% fechado.
- **Google (Gemini Live)** — tem o melhor *método* de latência (SoundStorm, geração não-autoregressiva paralela, ~100x mais rápida). Full-duplex real, mas fechado. Sotaque regional pt-BR não documentado.
- **OpenAI (GPT-4o Realtime)** — full-duplex real de referência (server VAD + barge-in por cancel+truncate), ~800 ms v2v. Fechado, mas o protocolo é documentado.
- **Inworld** — a ameaça **técnica** mais relevante de aprendizado: código MIT, serving SOTA (~200 ms), GRPO publicado. Mas o backbone é **LLaMA** (licença Meta, **fora do nosso gate**) — oportunidade: reusar o método trocando o backbone por um base permissivo.

A conclusão de posicionamento, verbatim do nosso dossiê: *"Somos os únicos 100% open mirando full-duplex + emoção + sotaque regional pt-BR. Ninguém ocupa nosso quadrante (open + carioca controlável + conversacional)."* E o ajuste fino, que muda a cabeça do projeto: o wedge não é "ter carioca" — é **carioca controlável + intensidade + open + full-duplex + emoção**; e a peça *tecnicamente* mais difícil e única não é o sotaque isolado, é o **loop de full-duplex + re-síntese-com-pivô**.

PARTE IV — A trilha do software: o que já construímos

Agora a parte concreta: o que existe no repositório, organizado pela “trilha” que o próprio projeto desenha no `rate_app` (o cockpit) e no arquivo `tools/rate/trilha_map.json` — o mapa mental data-driven do projeto, com lanes, nodes, hipóteses (validadas/abertas/refutadas) e dependências.

12. As três trilhas + a transversal (o mapa)

O projeto corre em **três trilhas paralelas** mais uma camada **transversal**:

- **Trilha A — “A Voz”** (TTS expressivo finetunado, a sua voz). É onde mais avançamos. Roda no Colab/M2/RunPod. **Progresso ~46%**.
- **Trilha M — “Maya” (a cascata)**. O pipeline em `src/duplex` que reproduz a arquitetura da Maya (VAD→ASR→LLM→CSM). **Progresso ~28%**.
- **Trilha B — “A Conversa” (Moshi)**. O spine full-duplex nativo. **Parkeada** nesta rodada (aposta de médio prazo). **Progresso ~3%**.
- **Transversal** — Eval, Infra (esteira RunPod), e os Aprendizados. **Progresso ~60%**.

As trilhas não competem: o **dataset da sua voz serve às três**. A existência da Trilha M *sobe o peso estratégico do CSM* (é a peça central da reprodução da Maya). E há um **Gate F4** que decide o futuro: se a cascata (M) bater a meta de latência ($p50 < 800$ ms) e de preferência humana ($\geq$ Moshi-pt, em escuta cega com 3 cariocas), **M vira a abordagem principal** e a Trilha B é reavaliada. Se falhar, volta-se ao Moshi.

13. Fundação: os datasets e o problema dos dados pt-BR

O gargalo do projeto **não é modelo, é dado**. E “dado” tem duas dimensões que se confundem: **quantas horas existem** (volume) e **que tipo de horas, e se você pode usá-las legalmente** (registro + licença). A lição estrutural: **dado LIMPO e legalmente utilizável vale mais do que muitas horas sujas ou bloqueadas**.

Dois conceitos-base:

- **Fala lida (read speech) vs. espontânea (spontaneous)**. Áudio *lido* é alguém narrando texto pré-escrito (audiobook) — limpo, articulado, ótimo para aprender a *pronunciar bem*, péssimo para aprender a *conversar*. Fala *espontânea* tem hesitação, risada, overlap, prosódia natural — é o que a Maya faz e o que falta em pt-BR aberto. Aprender a recitar um poema decorado é diferente de aprender a bater papo no boteco. O produto quer o segundo.

- **A licença dura do projeto:** só **Apache-2.0 / MIT / CC-BY / CC0** entra no produto. **NC** (non-commercial) mata o produto; **ND** (no-derivatives) também mata, porque **treinar um modelo é, por definição, criar um derivado**. A primeira pergunta em cada corpus não é “quantas horas?”, é “posso colocar isso num produto pago?”.

O inventário pt-BR, resumido:

Corpus	Horas pt	Registro	Licença	Entra no produto?
CML-TTS	~68 h	lido (audiobook)	CC-BY-4.0	Sim — seed de inteligibilidade
MLS-pt	~161 h	lido	CC-BY-4.0	Sim (mas tende a pt-PT)
Common Voice pt	~187 h validadas	lido (crowd)	CC0	Sim — melhor diversidade de sotaque
TTS-Portuguese	~10,5 h	lido	CC-BY-4.0	Sim
TAGARELA	8.972 h	podcast espontâneo	CC-BY-NC-SA	NÃO (eval-only) — o gigante vetado
CORAA / NURC / CETUC	centenas de h	espontâneo	NC / ND / pesquisa	NÃO
Voz do Pedro	~48 min hoje	conversacional carioca	nossa	Sim — o moat

Uma correção importante que fizemos na fonte primária: durante um tempo a doc dizia que o **CML-TTS tinha ~1.100 h de português** — na verdade, **eram ~68 h**; o número grande era a linha do **alemão** na tabela do paper. Confirmamos pelo tamanho do arquivo (9,7 GB não cabem 1.100 h a 24 kHz). Lição de método: quando um número parece grande demais, **cheque uma grandeza ortogonal** (aqui, bytes no disco).

O ponto que dói: **todo o volume grande e todo o espontâneo de verdade está do lado vetado** (NC/ND). O pool limpo (CC-BY/CC0) de fala **lida** soma a ordem de ~430 h e **resolve pronúncia, não conversa**. E **emoção pt-BR comercial-safe ≈ 0 h** — esse é o moat que ninguém ameaçou. Por isso a aposta num **flywheel próprio**: gravar a sua voz dirigida + capturar reuniões com consentimento. É a única fonte que é, ao mesmo tempo, limpa, conversacional, carioca e 100% nossa em direito.

A ação de maior alavancagem aqui custa um e-mail. O NILC/TaRSila (USP) publicou derivados do NURC para TTS (`NURC-SP_ENT0A_TTS` espontâneo, e `nurc_tts_24khz` com subset **Recife** — primeiro corpus TTS nordestino em escala) com uma **tag de licença MIT** que conflita com o upstream NC-ND. Se o NILC confirmar que a tag é intencional, ganhamos de uma vez a **primeira fala espontânea pt-BR legal** no produto. Por isso “e-mail ao NILC/Frederico Oliveira” é o item nº 1 da sua checklist.

O KIT DE GRAVAÇÃO: A CIÊNCIA DE GRAVAR A SUA VOZ

Há um kit completo em `tools/recording/`. As regras não são frescura de áudio — cada número tem uma razão, e duas teses governam tudo:

1. **O modelo aprende a SALA junto com a voz.** Um TTS é um aproximador estatístico: ele copia *tudo* que é consistente. Se toda gravação tem o mesmo eco de parede, ele decide que o eco *É* a sua voz. Daí a regra de ouro: **consistência acima de perfeição** — mesma sala, mesma posição do mic, mesmo ganho em todas as sessões.
2. **O gargalo não é volume, é cobertura.** 24 h de fala lida monótona ensinam menos que 6 h que cruzam emoções, sotaques e diálogo real.

Os números sagrados da captação e o porquê de cada um:

- **SNR $\geq$ 32 dB** (signal-to-noise ratio) — abaixo disso o ruído de fundo vira parte do timbre. Critério herdado do Hi-Fi TTS da NVIDIA.
- **48 kHz / 24-bit / mono** — captura todo o espectro audível com margem de dinâmica; o Mimi consome 24 kHz de qualquer forma.
- **Picos entre -12 e -6 dBFS** — forte o suficiente para ficar longe do ruído, com folga para não **clipar** (encostar em 0 dBFS = distorção irreversível).
- **-23 LUFS só no export, nunca peak-normalize.** Normalizar por *loudness percebido* (LUFS) iguala o volume médio de todas as frases (dataset homogêneo); normalizar por *pico* deixa uma frase com um estouro baixinha no resto (o modelo aprende a variar volume aleatoriamente).
- **Sem denoise agressivo.** Denoisers deixam artefatos que o TTS aprende — você troca um ruído por outro pior, colado na voz. Invista em **captar limpo na origem**.
- **Cobertura fonética (49 fones, trifones).** O modelo precisa ter ouvido você pronunciar todos os sons em contextos variados — senão “chuta”. A única lista calibrada na distribuição **do Rio** (Alcain 1992) vira o script-base.
- **”4 h de trabalho $\approx$ 1 h de áudio útil”** — número que precisa estar no seu orçamento mental. O dataset completo (~25–40 h úteis) é um projeto de centenas de horas de trabalho, não um fim de semana.

Os **cartões de emoção** (8 estilos $\times$ 3 intensidades, com frases-âncora idênticas em todos os estilos — “ouro” porque criam *minimal pairs* onde só a emoção muda) e os **cartões de sotaque** (5 sub-variações tratadas como *estilos*, não dialetos separados, com eval cega para não virar caricatura) completam o kit.

E o **flywheel de reuniões**: gravar as reuniões da Unflat (você + João, cariocas, + Guilherme, paulistano) com **1 canal por pessoa** vira uma esteira de dado conversacional real, espontâneo, com overlap e backchannel — o mais caro de produzir artificialmente. O hardware exige **microfones dinâmicos** (não condensadores — condensador capta a sala toda, e o *crosstalk* entre canais arruína a separação por falante) e **uma interface multicanal** (não 3 USB — três clocks independentes derivam e dessincronizam os canais). Consentimento LGPD de João e Guilherme **antes** da primeira gravação.

14. Trilha A — A Voz: a receita de dois estágios e a jornada de treino

Esta é a trilha mais avançada, e a que produziu o **Treino 1**. A arquitetura do CSM (Parte 6) dita a estratégia: **dois estágios**, porque adaptar **língua** (estrutura semântica) e clonar **voz** (timbre acústico) são problemas diferentes.

14.1 A DESCOBERTA QUE ANCOROU TUDO: “VOZ É BARATA, LÍNGUA É O INVESTIMENTO”

Antes do treino sério, um *smoke test* local no seu M2 (csm-mlx, LoRA, 48 min da sua voz da ElevenLabs, 2 épocas) deu um resultado de duas faces:

-  **A identidade de voz ENTROU NOS PESOS.** Sem nenhuma âncora de contexto, o modelo já gerava com o seu timbre em nível de teto (spk-sim 0,969–0,980; o teto real-vs-real é ~0,965). **48 minutos bastam para a voz.**
-  **O português não** (WER 100%+, “português-aparente” sem conteúdo). Coerente com a evidência de outras línguas: ensinar **língua** exige ~40x mais exposição do que demos ali.

Isso reforçou a receita: **Estágio A** (ensinar pt-BR ao CSM com corpus aberto) → **Estágio B** (LoRA da sua voz por cima). E provou que **a infra roda ponta a ponta num MacBook** (transcrição → dataset → treino → adapter → inferência).

14.2 ESTÁGIO A — ENSINAR PORTUGUÊS AO CSM

O CSM nasce treinado em inglês. O Estágio A é um finetune de **língua** usando o **CML-TTS** (68 h de audiobook formal pt, CC-BY). A configuração que funcionou (e o caminho até ela) é uma aula de hiperparâmetros:

- **Learning rate 5e-4** (não 2e-4) + **180 min** de treino (não 60) → **WER 21%** (o vencedor da bateria, batizado de **BASE-PT**). Com LR 2e-4 e 60 min, o WER era 116%.
- **LoRA r=64 / α=64**, scheduler cosine, ~14 épocas.

O número WER 21% basta para inteligibilidade em português. Esse modelo vive em `runs/battery_A1_cml/final` e é a base sobre a qual a sua voz é construída.

14.3 ESTÁGIO B — A SUA VOZ POR CIMA DA BASE-PT

Aqui se carrega a BASE-PT, faz-se o merge dela no modelo, e treina-se um **LoRA novo** com as 362 frases da sua voz (ElevenLabs). Duas lições caras:

- **Learning rate baixo (5e-5) é crucial.** Com LR 2e-4, o modelo **overfittou** (WER 300% — memorizou e destruiu o português). O overfit anterior era hiperparâmetro, não dado.
- O resultado **final** (com o fix do EOS, abaixo, + 5e-5 + 90 min): **WER 12% round-trip e 14/14 frases parando sem balbuciar.** O adapter vive em `runs/stage_b_final/final` (e em `models/stage_b_final_adapter/` no repo).

14.4 OS GOTCHAS DE TREINO — OS BUGS CAROS QUE VIRARAM RECEITA

A “jornada” de 15–17 de junho rendeu uma lista de armadilhas que hoje estão **codificadas em guardrails** (`runpod/recipe.py` , função `check_config` , que aborta cedo se a config repete um erro conhecido). Vale entender cada uma, porque elas ensinam como esses modelos quebram:

1. **Warmup time-capped.** Use `warmup_steps` **fixo** (ex.: 20), **nunca** `warmup_ratio` . Com `num_train_epochs=99` , um ratio de 0,03 vira ~1.188 steps de warmup, mas o time-cap para a run em ~300–540 steps → a run inteira fica com learning rate ≈ 0 e **não aprende** (loss travada, WER 185%). Sintoma: loss não desce + learning_rate minúsculo no log.
2. **O balbúcio (o gotcha mais caro) — o fix do EOS.** O CSM **para** quando emite um frame todo-zero (`[0]×32` codebooks). O token `<|audio_eos|>` (128003) é só um marcador de texto e ficava com label -100 → o frame de parada **nunca** era alvo de loss → o modelo **balbuciava até o limite de tokens**. O fix (1 linha): setar o label do `<|audio_eos|>` para **0** (o frame de silêncio). Resultado: de **0/14 frases parando** para **14/14**, e o WER despenca (sem cauda de lixo). **Não** setar para 128003 (estoura o vocabulário do depth decoder → crash CUDA). Esse é o tipo de detalhe que só se descobre lendo o código de geração (`generation_csm._sample`).
3. **Áudio real, não zero-pad.** Cortar o áudio na mão (`array[:288000]` = 12 s) e usar áudio real com cutoff real + collator por-batch — senão o modelo aprende a “encher 12 s” de silêncio.
4. **Texto longo trunca o áudio.** `max_text_len` baixo (< 256) faz texto longo (>94 tokens, ~4% do TAGARELA) trincar os placeholders de áudio → shape mismatch → crash. Esse foi um bug que só aparecia em **dados representativos** — um smoke-test de 30 cliques não pegava (só 4% têm texto longo). Lição registrada na memória do projeto: **confirmar a causa-raiz e testar com amostra representativa antes de gastar GPU**.
5. **GPU data-starved sem dataloader workers.** Com `num_workers=0` , a utilização da H100 ficava picotada (0%↔92%, média ~21%) — a collation de áudio rodava no thread principal entre os steps, e a GPU esperava. O fix (`dataloader_num_workers=8` + `pin_memory` + `prefetch`) levou a util média para ~57% com picos sustentados de 90–100%, e o step de 2,7 s → 1,45 s (~2×).

14.5 O TREINO 1 — O QUE ELE PROVOU

Em 17 de junho de 2026, você avaliou 42 amostras no `rate_app` . O veredito:

Modelo	Geral	Soa nativo	Natural	Voz	Para	WER
cml_long (base pt)	2,6	2,3	2,6	—	79%	—
stage_b (v1, sujo)	1,7	2,1	1,4	2,7	36%	73%
stage_b_final	3,1	2,8	3,1	3,4	93%	12%

As conclusões:

- **✓ A sua voz/timbre está PROVADA.** O `stage_b_final` tem voz 3,4/5, para 93%, WER mediana 12%. O moat funciona — **não mexer nela.**
- **✗** O gap agora é **SOTAQUE** (“gringo” é o problema nº1, marcado 28x; “soa nativo” ~2,8/5 mesmo no melhor), **PROSÓDIA robótica** (2º maior, 18x) e **leitura de NÚMERO** (CEP/protocolo/R\$ quebram — isso é problema de *front-end de texto*, não de voz).
- O sotaque carioca **quase não transfere** (suspeita: a base CML formal/paulista domina o sotaque sobre a sua voz).

O que “WER 12%” significa, com precisão: é o WER round-trip avaliado no benchmark de 14 frases (gera áudio → ASR → compara com o texto-alvo). O treino em si foi nas 362 frases da sua voz — os 12% **não** são sobre as frases de treino. E o WER **não captura sotaque** (fonemas mal pronunciados que ainda formam palavras): no Treino 1, o “gringo” foi marcado 28x e o WER não pegou. Por isso o eval perceptual (Parte 17) é tão importante.

15. Trilha M — Maya-BR v0: a cascata, peça por peça

A cascata vive em `src/duplex/`. **Status honesto: scaffold** — a arquitetura está ligada e cada peça foi validada isoladamente, mas a corrente **nunca rodou de ponta a ponta com mic + GPU reais**. O pipeline:

```
mic → turn_engine (silero-VAD + smart-turn) → asr (faster-whisper)
    → llm (endpoint OpenAI-compatible, streaming por sentença)
    → tts_adapter (pocket | chatterbox-ptbr | csm | qwen3) → playback interrompível
```

`turn_engine.py` — o ouvido e o porteiro. Tem a inteligência de turn-taking em dois estágios:

- **VAD (silero)** decide “tem voz?” a cada frame de 32 ms.
- **SmartTurn v3** (Pipecat, BSD-2; ONNX de 8 MB, 12–95 ms em CPU, **95,4% de acurácia em português**) decide a pergunta sutil: aquele silêncio de ~280 ms é **fim de turno** ou só uma **pausa para pensar?** (Cortar na pausa é o erro clássico das assistentes que te interrompem.) Se o SmartTurn disser “pausa”, um fallback duro de 600 ms eventualmente fecha o turno.
- O **Player** toca o áudio em blocos de **80 ms** num thread separado, com uma flag de parada — daí o **barge-in** corta em ~80 ms quando você fala por cima (só com fones; em caixa de som usa half-duplex por causa do eco, com um cooldown anti-eco de 350 ms).

`asr.py` — faster-whisper “small”, `language="pt"`, `beam_size=1` (latência mínima). WER baseline 21%.

`llm.py` — wrapper **OpenAI-compatible** (Gemini, Maritaca/Sabiá, sglang, ollama — trocar o cérebro = trocar duas strings). O truque de latência é o **streaming por sentença**: assim que uma frase completa aparece no stream do LLM, ela já é mandada ao TTS, enquanto o LLM ainda escreve o resto. E há `mark_interrupted` (corrige o histórico pós-**barge-in** para o LLM não “achar que falou” o que foi cortado).

`tts_adapter.py` — interface única, vários engines. O coração é o **CSMAdapter**, que materializa o “segredo Maya”: ele empilha o **áudio** dos turnos (seu e do agente, roles “1” e “0”, janela de 4 turnos) e a fala que acabou de gerar **vira contexto** para o próximo turno. É o audio-conditioning rodando na prática. Há também o `CSMMLXAdapter` (roda no seu M2 via MLX) com a descoberta das ≥ 3 âncoras.

`chat_loop.py` — o maestro, que encadeia tudo e **loga as latências** (asr, llm₁, tts₁, total→1ºaudio) — exatamente as métricas que alimentam o Gate F4.

O que falta (scaffold honesto): rodar fim-a-fim com mic+GPU; **tocar por sentença** (hoje concatena e toca de uma vez — um TODO que anula parte do ganho do streaming); **fechar o ciclo de barge-in no maestro** (o `mark_interrupted` e o `truncate_last_agent` existem, prontos, mas o loop ainda não os chama). E uma limitação de fundo: barge-in hoje é “detectar novo turno”, não **re-síntese incremental** (cortar a geração em voo, estilo Sesame com abort de 20 ms) — isso é Fase 5.

16. Trilha B — A Conversa (Moshi): por que e quando

O **Moshi** é a aposta de longo prazo para o spine full-duplex nativo (Parte 6.2). Nesta rodada ela está **parkeada** — a receita foi **lida e entendida**, mas **nada rodou em pt-BR ainda**. As razões:

- O ganho do Moshi sobre a cascata só aparece com **dado conversacional estéreo em escala** (o fly-wheel de reuniões), que ainda não existe.
- **J-Moshi/LLM-jp-Moshi** já provaram que a transferência de língua funciona (e pode sair Apache); **moshika-rl-seamless** já entregou o RL de interatividade. O caminho está pavimentado, então não há pressa de pioneirismo.
- Surgiu um concorrente do próprio Moshi no nosso caminho crítico: o **SoulX-Duplug** (módulo plug-and-play que daria “duplex” sobre a cascata Maya **sem treinar um spine do zero**) — a avaliar antes de comprometer com o Moshi.

A regra operacional: **reabrir a Trilha B só se o Gate F4 da cascata (M) falhar**. O notebook do moshi-finetune está escrito (`notebooks/5_moshi_spine.ipynb`), a receita está documentada (`runpod/RUN-BOOK-moshi.md`), e o smoke do round-trip do Mimi em pt-BR está em `phase0/spike_c_moshi/`. É um míssil pronto na rampa, esperando o dado.

17. Transversal: eval, o compasso (rate_app), a esteira e os aprendizados

17.1 O EVAL HARNESS — COMO MEDIMOS SE A VOZ FICOU BOA

Avaliar TTS é diferente de ML clássico: não há um “certo” único. A régua em inglês mente em pt-BR (UT-MOS descalibrado, WER satura, spk-sim sem escala absoluta), então **construímos a nossa régua em camadas** — barata e automática embaixo, cara e humana no topo:

- **WER round-trip** — gera áudio → ASR transcreve → compara com o texto fonte. Mede **inteligibilidade**. É “the core gate”, mas **satura** (todo TTS bom tira nota perto de zero) — por isso usamos **dois ASRs** (whisper-large-v3 + Parakeet) e uma sonda de pronúncia que **não satura** (homógrafos: *sede* d’água vs. da empresa).
- **Speaker similarity (spk-sim)** — cosseno entre embeddings de voz (WavLM-SV): “ainda soa como o Pedro?”. Reportar **sempre** com o teto real-vs-real (~0,965 no nosso setup) — porque nem dois cliques da mesma pessoa dão 1,0. O gate de fase é **spk-sim ≥ 0,70**.
- **TTSDS2** — a métrica **distributiva** principal desde jun/2026: compara a *distribuição* acústica do que geramos com a da fala real (não precisa de notas MOS, só de fala real de referência — que temos). Foi a única (de 16 métricas) com correlação de Spearman > 0,5 contra MOS humano em todos os domínios.
- **DNSMOS** — higiene de áudio (gate de gravação). **UTMOS** — rebaixado a “número histórico” (não calibrado pt, instável entre runs).
- **CMOS / MUSHRA humanos** — o ground truth final, no gate de release (≥ 30 ouvintes BR, cego). **SER pt-BR** — para emoção (a construir). **BIPA** — pronúncia dialetal carioca.

E o “**Maya parity checklist**” (`eval/maya_parity.md`): a Trilha M compara **sempre contra a Maya real** (você tem o app iOS) — latência percebida (resposta de turno ≤ Maya +20%, barge-in ≤ 300 ms), comportamento conversacional (backchannel, overlap, prosódia contextual, memória — pontuado /14), e sotaque (escuta cega com 3 cariocas). Um bloco decisivo separa **voz** de **cérebro**: respostas sem graça são culpa do **LLM plugável**, não da voz — e “trocar o LLM não pode exigir retreinar a voz”.

Construir esse eval não é overhead. **Não existe benchmark TTS pt-BR público** — então a nossa régua é um **fosso reputacional** e um paper futuro (PROPOR/Interspeech 2027).

17.2 O RATE_APP — O COMPASSO DO PROJETO

`tools/rate/rate_app.py` deixou de ser um classificador simples e virou o **cockpit do projeto** — um app local (stdlib, identidade visual Unflat). Tem três abas:

- **Avaliar** — notas estruturadas (geral, soa-nativo, natural, voz, parou) + tags de problema pt-BR-aware (R /ʌ/, vogal nasal, ti/di palatal, S coda carioca, etc.).
- **Insights** — agrega notas por run e emoção, rankeia os problemas (qual defeito atacar primeiro).
- **Trilha** — o mapa mental interativo (data-driven via `trilha_map.json`), com nodes, dependências, % e deep-dives clicáveis.

A virada estratégica: **não é sobre WER — é sobre acumular feedback humano estruturado e localizado no tempo** como substrato para os **futuros loops de agentes** que vão corrigir cada erro pontual no próximo treino. Dois tipos de erro são capturados: **objetivo** (WER decomposto em troca/omissão/insersão por palavra) e **perceptual** (markers no tempo via waveform clicável). O contrato está versionado em `tools/rate/FEEDBACK.md`. “Não construir os agentes agora, só preparar o terreno.”

17.3 A ESTEIRA RUNPOD — A 2ª PISTA DE TREINO

Além do Colab, montamos uma 2ª esteira: um pod **RunPod H100 80GB** que o Claude **dirige direto via SSH** a partir do seu Mac (loop de debug rápido, sem você relayar erro). Tem um trio de scripts: **watch-dog** (escreve `status.json` a cada 30 s, detecta stall/running/done/crashed, limpa disco se $\geq 92\%$), **overnight** (orquestra o grid de runs com timeout/deadline) e **aggregate** (monta o relatório, ordena por WER, destaca o melhor). Valor: treino offline autônomo, **~3,5x mais barato que Colab Pro+** para velocidade. Custo ~\$3,29/h — com a disciplina de dar **Stop** quando terminar.

17.4 OS APRENDIZADOS COMO ATIVO

A maratona de treino virou conhecimento codificado: warmup time-capped, streaming `decode=False` (lê só os bytes, decoda na tokenização — libera GPU ociosa), áudio real, EOS label=0, Stage B com LR baixo. Tudo isso vive em `runpod/recipe.py` (a receita validada + guardrails) e na memória persistente do projeto. **Método eficaz: revisar e fazer smoke-test ANTES de gastar GPU** — porque cada re-run errado queima dinheiro e tempo.

PARTE V — Onde estamos e para onde vamos

18. O que o Treino 1 provou — e o que falta

Em uma frase: **a voz está provada; o gap inteiro é o “soa nativo”**. O timbre carioca entrou nos pesos, o modelo para de falar na hora certa, e a inteligibilidade (WER 12%) é boa. O que falta para soar como gente do Rio:

1. **Sotaque “gringo”** (problema nº1) — fonemas mal pronunciados que o WER não pega. Ataque em duas frentes separadas: **fonético** (G2P) e **prosódico** (proporção de dado/gravação).
2. **Prosódia robótica** (nº2) — vem em parte da mono-emoção do dataset atual; resolve com o G2 (emoções gravadas).
3. **Leitura de número** — problema de **front-end de texto**, não de voz. Já temos o normalizador.

A boa notícia: **a maior parte do que move “soa nativo” é mais barata do que re-treinar**. Começa pelos quick-wins sem GPU.

19. O roadmap priorizado e o plano de GPU

A ordem, do barato ao caro:

1. **Normalizar número/CEP/moeda/% no front-end** — *já feito*, sem GPU. O módulo `tools/text/normalize_ptbr.py` expande “R\$ 1.350,90” → “mil trezentos e cinquenta reais e noventa centavos” e “22290-160” dígito a dígito, **antes** do CSM, no treino E na inferência (o “ponto de injeção único” de texto, onde o G2P também vai entrar). É determinístico, testável, roda na CPU.
2. **Baseline limpo** — rodar o `curate_app` (corrigir as ~5–10% de transcrições erradas do Whisper, descartar fragmentos/ruído) e re-treinar o Estágio B no dataset curado. **Este é o único bloqueador 100% nosso** — sem ele, todo experimento fica contaminado. (*Pendência: o `curate_app` ainda não rodou; você até agora só **classificou** os áudios no `rate_app`, que é outra etapa.*)
3. **G2P pt-BR** (fator fonético do sotaque) — fonemizar a entrada, como um braço de ablação do Estágio B v2. Engine pronto sem depender do BIPA: CharsiuG2P ou LatPhon (ambos MIT).
4. **Base carioca / ablação de proporção** — testar se mais da sua voz cedo (30/70, 50/50, 100%) deruba o gringo e recupera o carioca. NURC só depois do NILC confirmar a licença.
5. **enrich_markers (GOP/MDD)** — forced-alignment offline (CPU, custo zero) sobre o Treino 1, para transformar “soa gringo” (nota global, 28x) em **fonema-errado-no-tempo**.
6. **Gravar o G2 (emoções)** — 8 estilos × 3 intensidades, ~5–7 h. O caminho **único** para emoção (task-vector não funciona — Parte 8). Por último, porque é o mais caro.

O plano de GPU (em `trilha_map.json`, `gpu_plan`): o Treino 1 custou **~\$40** (~12 h de H100 80GB @ \$3,29/h — Estágio A ~3 h + grid de ~8 runs + 4x Estágio B). O próximo sprint do sotaque (B2-limpo + B2-G2P + ablação de proporção) **≈ 9 h ≈ \$30 no H100** (ou ~\$14 no A100, mais lenta mas barata). Os itens #1 (normalizador, feito) e #5 (forced-align, CPU) não contam. Cada novo modelo vira um bloco (Treino 2...) no `rate_app`.

20. Os princípios de trabalho

Por fim, as regras de método que o projeto adotou — porque elas explicam *como* trabalhamos, não só *o quê*:

- **Áudio é o core, não texto.** Checar a fonte primária (paper/model card) em decisões de arquitetura e licença, nunca um resumo.
- **Fase é research-only.** Qualquer dataset (mesmo NC) pode entrar nos **experimentos** — o que importa é a **rastreabilidade** (qual dado treinou qual peso, no nome do run + em `specs/EXPERIMENTS.md`). O caminho de retreino limpo (CC/próprios) para o produto fica sempre documentado. O risco de licença é de *shipping*, não de experimentar.
- **Revisar antes de rodar.** Confirmar a causa-raiz do bug + smoke-test com amostra **representativa** (incluindo casos de borda) antes de lançar uma run que custa GPU. Aprendido na pele (o bug do texto longo que só aparecia em 4% dos dados).
- **Licenças.** Só Apache/MIT/CC-BY/CC0 no produto. Vetados: XTTS (CPML), F5-TTS (NC), CORAA/NURC (NC-ND). Câmara **ao vivo** é CC-BY (ok); novela/filme = **veto total**.
- **Compliance-by-design.** Consentimento + watermark desde o dia 1 — já alinhado ao PL 1460/2026.
- **Termos técnicos em inglês, explicação em português.** Porque o material de referência (papers, repos) é todo em inglês, e traduzir o jargão atrapalha a busca e o aprendizado.

Fecho: a tese em uma respiração

Estamos construindo a **única** voz conversacional pt-BR que é, ao mesmo tempo, **aberta, carioca controlável, full-duplex e emocional** — um quadrante de mercado que ninguém ocupa. A arquitetura que escolhemos (native audio, Mimi/RVQ, CSM + Moshì) é **a mesma da Maya**, só que aberta e brasileira — e a Maya, descobrimos, **não tem segredo nem patente**: é orquestração + dados + engenharia, tudo replicável. Já **provamos a sua voz** (Treino 1: timbre 3,4/5, WER 12%, para 14/14). O que falta — soar nativo — é em boa parte mais barato que re-treinar: normalizar texto, curar o dataset, fonemizar a entrada, e gravar a emoção na sua própria voz. O **moat** não é o modelo (o CTO da Sesame admite que “even the 1B is very good”); é a **persona + o sotaque + a latência + o dado próprio + o gosto de quem monta**. E esse dado, essa voz e esse gosto são seus.

Glossário

- **ASR** (*Automatic Speech Recognition*) — fala → texto. **TTS** — texto → fala. **VAD** — detector de “tem voz?”.
- **Full-duplex / half-duplex** — falar e ouvir ao mesmo tempo (telefone) vs. um de cada vez (walkie-talkie).
- **Barge-in** — interromper o agente por cima, e ele cala na hora. **Backchannel** — “uhum/sei/caraca” enquanto o outro fala.
- **Cascata vs. native audio** — etapas separadas (VAD→ASR→LLM→TTS) vs. um modelo que pensa em tokens de áudio direto.
- **CSM** (*Conversational Speech Model*, Sesame) — o modelo de voz (1B, Apache-2.0); audio-conditioned; o nosso clonador. **Moshi** (Kyutai) — o spine full-duplex (CC-BY-4.0). **Mimi** — o codec neural (12,5 Hz) compartilhado pelos dois.
- **Codec neural / RVQ** — comprime áudio em tokens discretos; RVQ = pilha de codebooks, cada nível refina o resíduo do anterior. **Semantic tokens** = conteúdo/estrutura; **acoustic tokens** = timbre/identidade.
- **Backbone / depth decoder** — o transformer Llama (prevê frames) / o decoder que completa cada frame na profundidade dos codebooks.
- **Finetune / LoRA / adapter** — re-treinar com seus dados / finetune barato (treina um delta pequeno) / o arquivo resultante. **Zero-shot/in-context** — imitar de exemplos no prompt, sem treino. **Âncora** — esses exemplos de referência. **CPT** — re-treino pesado de língua. **Quantização** — comprimir pesos (16→8/4-bit).
- **SFT / DPO / GRPO** — treino supervisionado / alinhamento por preferência (pares) / RL por grupo (recompensa). **Reward hacking** — o modelo enganar a métrica em vez de soar bem.
- **G2P** (*grapheme-to-phoneme*) — texto → fonemas (IPA). **GOP/MDD** — score de pronúncia / detecção de fonema errado no tempo (o “sotaque de gringo” acadêmico).
- **WER** (*Word Error Rate*) — % de palavras erradas; usado em **round-trip** (TTS→ASR→compara). “Satura” quando todo modelo bom tira nota perto de zero. **spk-sim** — cosseno de embeddings de voz (“é a mesma pessoa?”). **TTSDS2** — métrica distributiva vs. fala real. **MOS/CMOS/MUSHRA** — notas humanas. **SER** — reconhecimento de emoção na fala.
- **RTF / TTFA** — fator tempo-real (síntese ÷ duração) / time-to-first-audio (latência até o 1º som). **p50/p95** — mediana / pior caso típico (95%).
- **SNR / LUFS / dBFS / clipping** — razão sinal-ruído / loudness percebido / nível relativo ao teto digital / distorção por estourar o teto.
- **Read vs. spontaneous speech** — fala lida (audiobook) vs. espontânea (conversa). **Licenças**: CC0 (domínio público) / CC-BY (uso comercial com crédito) / NC (veta comercial) / ND (veta derivados — treinar é derivar).
- **Moat / wedge / flywheel** — fosso competitivo / a fatia estreita de entrada no mercado / a esteira que se retroalimenta (aqui, gravar reuniões gera dado que melhora o modelo).
- **OSINT** — investigação só com fontes públicas. **Watermark** — marca d’água inaudível de proveniência. **Compliance-by-design** — projetar já em conformidade (consentimento + watermark).